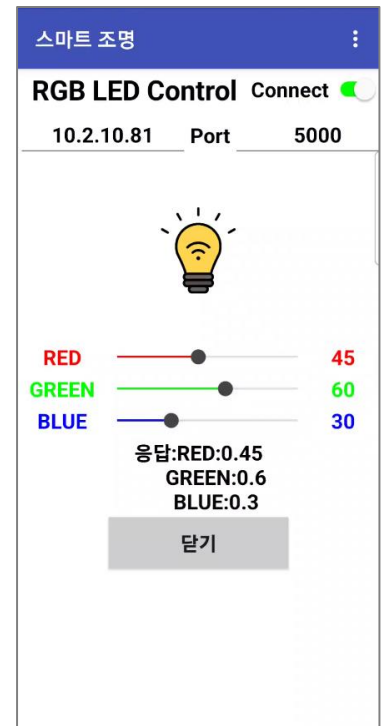




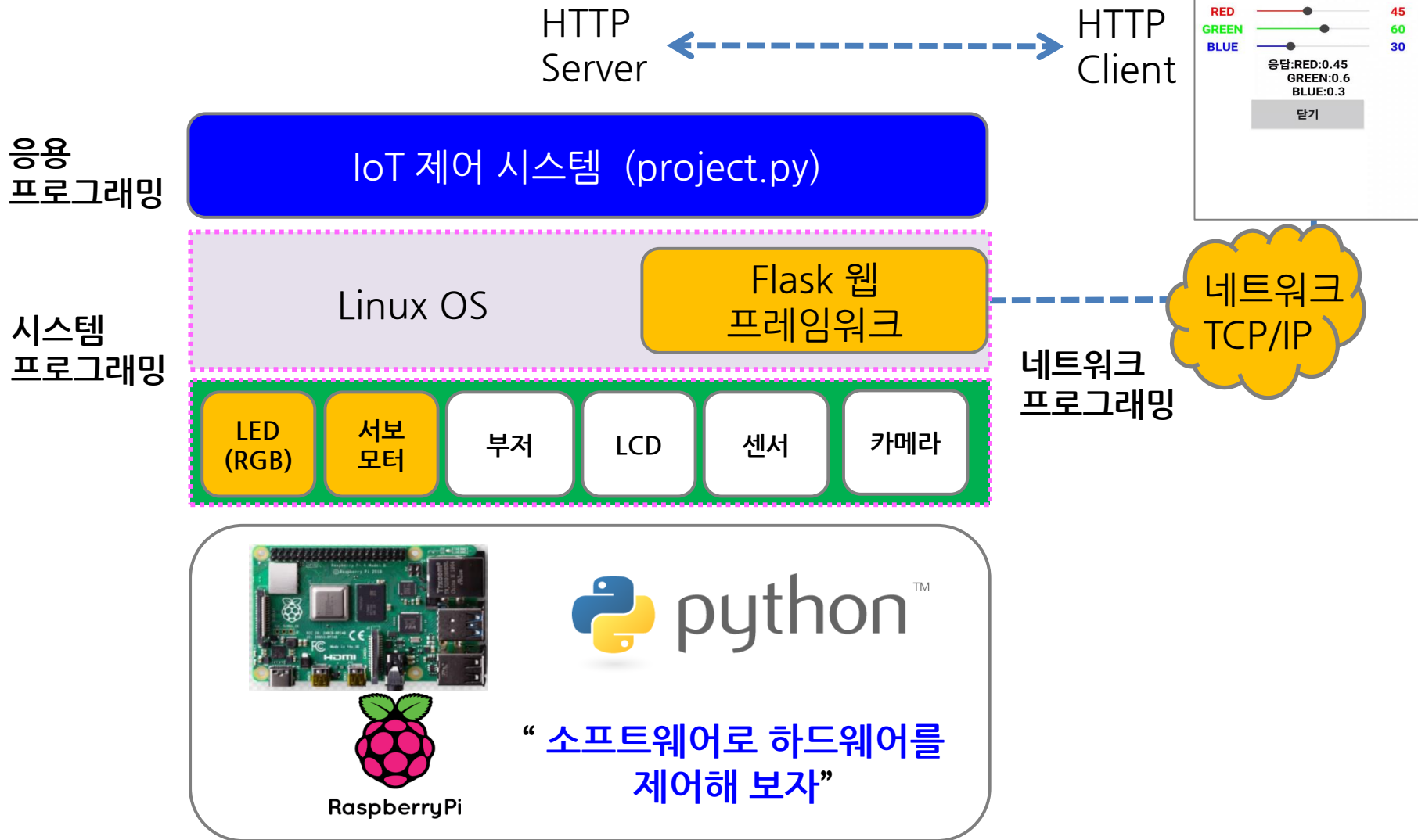
“ 소프트웨어로 하드웨어를 제어해 보자 ”

IOT 실습심화

6주차 : 스마트 조명



6주차 수업은 스마트 조명



* IoT 기술 요소 : 센싱 기술, 네트워킹 기술, 서비스 인터페이스 기술

5주차정리) Python Flask Web Framework

■ Flask 기초



- Flask는 파이썬 웹 프레임워크
 - Flask는 파이썬을 기반으로 한 마이크로 웹 프레임워크
 - django는 파이썬을 기반으로 한 풀 웹 프레임워크
- 웹 프레임워크(Framework: 틀, 도구상자)란?
 - 웹 애플리케이션 개발을 쉽고 빠르게 할 수 있도록 미리 만들어진 구조와 도구를 제공하는 소프트웨어
- Flask는 기본적으로 HTTP 프로토콜을 사용하며, HTTP는 TCP를 기반으로 동작
- 라즈베리파이에 Flask 설치를 설치하기

`pip install flask`

설치 에러 발생시) `pi@raspberrypi:~ $ pip install flask`
`error: externally-managed-environment`

```
pi@raspberrypi:~ $ sudo rm /usr/lib/python3.13/EXTERNALLY-MANAGED
pi@raspberrypi:~ $ pip install flask
```

5주차정리) Flask 기본 코드

■ 실습1) 라즈베리파이 Flask 웹 서버 기본 코딩

- Flask 애플리케이션을 생성하기 위한 기본 코드

flask_demo.py

```
# Flask 웹 프레임워크 기본 사용법
from flask import Flask

# 1. Flask 웹 서버 객체 생성
app = Flask(__name__)

# 2. 라우팅 설정: 어떤 주소(경로)로 요청이 있을시 어떤 함수가 실행될지 지정
# 사용자가 루트('/') 홈페이지에 접속 시 실행할 함수 지정
@app.route('/')
def home():
    return "<h1>Welcome Raspberry Web Server!</h1>"

# 3. 서버실행
# default는 IP주소는 로컬('localhost' 또는 '127.0.0.1'), port번호는 5000
if __name__ == "__main__":
    app.run(host='localhost', port=5000)
```

5주차정리) Flask Web Framework

■ 실습2) Flask 기초 : 라우팅 추가하기

- Flask 라우팅: 특정 URL 요청이 들어왔을 때 해당 함수가 실행되도록 URL과 실행 함수를 지정하는(매핑하는) 역할

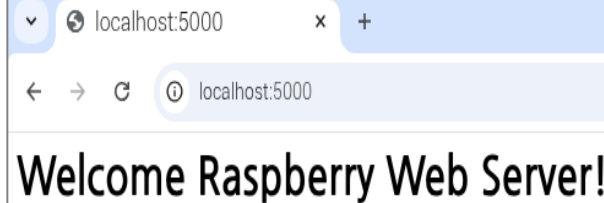
```
# Flask 웹 프레임워크 기본 사용법
from flask import Flask
```

```
# 1. Flask 웹 서버 객체 생성
app = Flask(__name__)
```

```
# 2. 라우팅 설정: 어떤 주소(경로)로 요청이 있을시 어떤 함수가 실행될지 지정
# 사용자가 루트('/') 홈페이지에 접속 시 실행할 함수 지정
@app.route('/')
def home():
    return "<h1>Welcome Raspberry Web Server!</h1>"
```

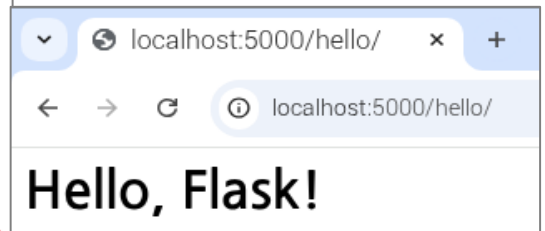
```
# 4. 경로추가: 사용자가 주소('/hello')로 요청시 라우팅 경로 추가하기
@app.route('/hello/')
def hello():
    return "<h1>Hello, Flask!</h1>"
```

```
# 3. 서버실행
# default는 IP주소는 로컬('localhost' 또는 '127.0.0.1'), port번호는 5000
if __name__ == "__main__":
    app.run(host='localhost', port=5000)
```



localhost:5000

Welcome Raspberry Web Server!



localhost:5000/hello/

Hello, Flask!

5주차정리) Flask GET 방식과 POST 방식 처리

■ 실습4) Flask GET, POST 메소드 처리 실습

아이디와 비밀번호를 서버측으로 전달해야 한다면

GET vs POST 방식 비교하기

두가지 방식으로 id와 pw보내고 URL 비교해 보기

1. GET 방식 로그인

아이디 비밀번호 GET으로 로그인

2. POST 방식 로그인

아이디 비밀번호 POST로 로그인

form 형식: 클라이언트가 서버로 특정 데이터를 보내는 양식

1. GET 방식 로그인

admin GET으로 로그인

URL - <http://localhost:5000/get/?id=admin&pw=1234>

※ 바디 사용 없이 URL에 쿼리 스트링으로 파라미터 데이터 전달

2. POST 방식 로그인

admin POST로 로그인

URL - <http://localhost:5000/post/>

`{ "id": "admin", "pw": "1234" } #body`

※ 바디를 사용해 파라미터 데이터 전달

5주차정리) Flask GET 방식과 POST 방식 처리

■ HTTP 요청 메소드 GET 방식과 POST 방식 비교

- GET 방식 요청과 POST 방식 요청의 차이

메소드	특징
GET	- 목적) 리소스를 얻어 오기 위한 요청 - 데이터 전달방식) 바디 사용 없이 URL에 쿼리 스트링으로 파라미터 데이터 전달
POST	- 목적) 새로운 리소스를 생성하기 위한 요청 - 데이터 전달방식) 바디를 사용해 파라미터 데이터 전달

id

pw

GET방식은 “http://localhost:5000/get/?id=admin&pw=1234”
쿼리스트링

POST방식은 요청메시지 body에 실어서 전송

body { “id”: “admin”, “pw”: “1234” }

5주차정리) Flask GET 방식과 POST 방식 처리

■ 실습4) Flask GET, POST 메소드 처리 실습(계속)

- GET 방식인 경우 'id='admin&pw=1234' 가 URL에 포함되어 쿼리 스트링으로 전달
※ Flask는 기본적으로 GET 방식

1. GET 방식 로그인

URL - <http://localhost:5000/get/?id=admin&pw=1234>

```
# GET 방식으로 로그인 하기, Flask는 기본적으로 GET 방식으로 동작
@app.route("/get/")
def login_get():
    # URL 쿼리스트링에서 (?id=value&pw=value) 데이터 가져오기
    user_id = request.args.get("id")
    user_pw = request.args.get("pw")

    if user_id == USER_DB["id"] and user_pw == USER_DB["pw"]:
        return f"""
        <h2>GET 방식 로그인 성공! 안녕하세요 {user_id}님</h2>
        <p>주소창(URL)에 입력한 id와 pw가 그대로 보입니다</p>
        <br><a href='/'>돌아가기</a>
        """
    else:
        return """
        <h2>로그인 실패!</h2>
        <p>아이디 또는 비밀번호가 틀렸습니다.</p>
        <br><a href='/'>돌아가기</a>
        """
```

GET 방식 로그인 성공! 안녕하세요 admin님

주소창(URL)에 입력한 id와 pw가 그대로 보입니다

[돌아가기](#)

로그인 실패!

아이디 또는 비밀번호가 틀렸습니다.

[돌아가기](#)

5주차정리) Flask GET 방식과 POST 방식 처리

■ 실습2) Flask GET, POST 메소드 처리 실습(계속)

- POST 방식인 경우 { "id": "admin", "pw": "1234" }가 body에 실려 전달
※ POST 방식을 처리하기 위해서는 라우팅에서 methods=["POST"] 추가 필요

2. POST 방식 로그인

URL - http://localhost:5000/post/

POST 방식으로 로그인 하기, methods에 POST 추가하기

```
@app.route("/post/", methods=["POST"])
```

```
def login_post():
```

요청 바디에서 (id=value, pw=value) 데이터 가져오기

```
user_id = request.form.get("id")
```

```
user_pw = request.form.get("pw")
```

{ "id": "admin", "pw": "1234" }

```
if user_id == USER_DB["id"] and user_pw == USER_DB["pw"]:
```

```
    return f"""
```

```
        <h2>POST 방식 로그인 성공! 안녕하세요 {user_id}님</h2>
```

```
        <p>주소창(URL)에 입력한 id와 pw가 보이지 않고 body에 실려 전송됩니다</p>
```

```
        <br><a href='/'>돌아가기</a>
```

```
    """
```

```
else:
```

```
    return """
```

```
        <h2>로그인 실패!</h2>
```

```
        <p>아이디 또는 비밀번호가 틀렸습니다.</p>
```

```
        <br><a href='/'>돌아가기</a>
```

```
    """
```

POST 방식 로그인 성공! 안녕하세요 admin님

주소창(URL)에 입력한 id와 pw가 보이지 않고 body에 실려 전송됩니다

[돌아가기](#)

로그인 실패!

아이디 또는 비밀번호가 틀렸습니다.

[돌아가기](#)

5주차정리) Flask 템플릿과 템플릿엔진 Jinja2

■ Flask 템플릿(template)과 템플릿엔진 Jinja2

- 템플릿(template)과 템플릿엔진 Jinja2 사용하는 이유

1) 템플릿으로 파이썬 로직 코드와 HTML 화면 디자인을 서로 분리하여 관리

- 파이썬 파일에는 데이터 처리나 하드웨어 제어 로직만 남기고, HTML 파일은 별도 관리

2) 템플릿 엔진을 통해 동적 웹페이지 제공 (Flask의 템플릿 엔진은 Jinja2)

- 정적 웹페이지와 동적 웹페이지의 차이는

"누구에게나 항상 똑같은 화면을 보여주는가, 아니면 상황에 따라 다른 화면을 보여주는가"

- 정적 웹 페이지: 서버에 미리 저장된 HTML 파일을 그대로 사용자에게 보여주는 페이지
(항상 똑같은 화면 제공)

- 동적 웹 페이지: 사용자의 요청, 시간, DB의 상태에 따라 실시간으로 HTML을 만들어 제공
(접속자, 시간, 입력 값에 따라 화면의 내용이 바뀜)

HTML + data처리 (파이썬 코드) 결합 → 동적 HTML

템플릿 엔진은 Jinja2의 기능

5주차정리) Flask 템플릿과 템플릿엔진 Jinja2

■ Flask 템플릿(template)와 템플릿엔진 Jinja2 사용법

- 템플릿(Template) : 사용자에게 보여줄 HTML 틀
- Jinja2: HTML 코드안에서 파이썬 문법(변수, 반복문, 조건문)을 결합해서 동적 HTML문서를 만들어 주는 Flask 템플릿 엔진 (<https://jinja.palletsprojects.com>)
→ 동적으로 웹 페이지를 생성하기 위함

- 프로젝트 구조

📁 프로젝트 폴더/

| |—— app.py (Flask 서버 코드)

|—— 📁 templates/

|—— index.html (Jinja2를 사용할 HTML 파일)

※ HTML 파일은 templates 폴더 밑에 저장

5주차정리) 템플릿 엔지 Jinja2 문법 - 변수

■ 실습6) Flask 템플릿엔진 Jinja2 사용법 - 변수

- 실습에 사용된 flask_demo.py 코드를 Jinja2 사용해서 동적 html을 작성해 보시다
- Jinja2는 템플릿 엔진 : HTML 문서안에서 파이썬 코드를 결합해서 동적 HTML 문서를 만드는 라이브러리

- Jinja2 변수 사용하기

{{ 변수명 }} # HTML에 변수 출력하기

Flask/

```
| |-----flask_template.py (Flask 서버 코드)
```

- └─ templates/
 - └─ hello.html (Jinja를 사용할 HTML 파일)

파일명: templates/hello.html

```
<!DOCTYPE html>
<html>
  <body>
    <h1>Hello, {{ name }}님! <br><br>
      Flask 템플릿 엔진 jinja2를 사용하여 변수를 반환합니다</h1>
  </body>
</html>
```

hello.html을 templates 밑에 저장

5주차정리) 템플릿 엔지 Jinja2 문법 - 제어문

■ 실습7) Flask 템플릿엔진 Jinja2 사용법 - 제어문

- Jinja2 제어문 사용하기

{% 제어문 %}

HTML에 제어문 사용

{% end제어문 %}

제어문 종료

◦ templates/fruits.html

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<!-- 변수는 중괄호 두개 사용 -->
```

```
<h1>안녕하세요, {{ name }}님!</h1>
```

```
<hr>
```

```
<h3>오늘의 과일 바구니</h3>
```

```
<!-- 제어문은 중괄호와 퍼센트(%) 사용, end로 닫기 -->
```

```
<ul>
```

```
{% for item in items %}
```

```
<li>{{ item }}</li>
```

```
<!-- for ~ else는 리스트가 비어있거나 없는 경우 실행 -->
```

```
{% else %}
```

```
<li>장바구니가 비어 있습니다</li>
```

```
{% endfor %}
```

```
</ul>
```

```
</body>
```

```
</html>
```

Flask/

| flask_template.py (Flask 서버 코드)

| templates/ (폴더 만들기)

| fruits.html (Jinja를 사용할 HTML 파일)

5주차정리) 응용실습1- Flask 템플릿을 이용한 웹 사이트 로그인

■ 응용실습1) flask_methods.py를 템플릿 구조로 변경하기

- 과제 수행방법

flask_methods.py는 파이썬 코드와 html 문서가 같이 있는 구조임.

이를 데이터 및 로직 처리부(파이썬 코드)와 웹 페이지 디자인(html문서)로 나누고 동적 웹페이지 제작이 가능하도록 템플릿과 템플릿엔진 jinja2를 사용해서 구현

1) 파이썬 웹 서버 코드는 주어져 있음 (단, 일부 코드가 누락되어 있으니 채우기)

2) html 문서는 2 종류

- login.html (홈 화면)은 이미 주어져 있으니 활용
- result.html(결과 화면)은 빈칸을 채워 완성하기

5주차정리) 응용실습1- Flask 템플릿을 이용한 웹 사이트 로그인

- 응용실습1) flask_methods.py를 템플릿 구조로 변경하기
 - 실행 화면 (로그인 요청:login.html과 로그인 결과:result.html 화면)

1. GET 방식 로그인

GET으로 로그인

2. POST 방식 로그인

POST로 로그인

GET 방식
id,pw일치

POST 방식
id, pw 일치

id, pw 불일치(공통)

← → ↻ ⓘ localhost:5000/get/?id=admin&pw=1234

Flask 템플릿과 템플릿엔진 jinja2를 사용한 로그인 결과

GET 방식 로그인 성공! 안녕하세요 admin님

주소창(URL)에 입력한 id와 pw가 그대로 보입니다

[돌아가기](#)

Flask 템플릿과 템플릿엔진 jinja2를 사용한 로그인 결과

로그인 실패

아이디 또는 비밀번호가 틀렸습니다

[돌아가기](#)

← → ↻ ⓘ localhost:5000/post/

Flask 템플릿과 템플릿엔진 jinja2를 사용한 로그인 결과

POST 방식 로그인 성공! 안녕하세요 admin님

주소창(URL)에 입력한 id와 pw가 보이지 않고 body에 실려 전송됩니다

[돌아가기](#)

5주차정리) 응용실습1- Flask 템플릿을 이용한 웹 사이트 로그인

■ 응용실습1) flask_methods.py를 템플릿 구조로 변경하기

1) 파이썬 웹 서버 코드 : 이름_5주차_응용실습1.py

```
from flask import Flask, request, render_template

# Flask 객체 생성
app = Flask(__name__)

# 테스트용 회원 정보
USER_DB = {"id": "admin", "pw": "1234"}

@app.route("/")
def home():
    # templates/login.html 파일을 찾아 전송
    return render_template("login.html")

# GET 방식으로 로그인 하기
@app.route("/get/")
def login_get():
    # URL 쿼리스트링에서 (?id=value&pw=value) 데이터 가져오기
    user_id = request.args.get("id")
    user_pw = request.args.get("pw")

    # 로그인 검증 후 성공 여부와 메시지를 result.html 템플릿에 전달
    if user_id == USER_DB["id"] and user_pw == USER_DB["pw"]:
        return render_template(
            "result.html",
            success = True,
            user_id = user_id,
            method = "GET",
            message = "주소창(URL)에 입력한 id와 pw가 그대로 보입니다"
        )
    else:
        return render_template("result.html", success=False)
```

```
# POST 방식으로 로그인 하기
@app.route("/post/", methods=["POST"])
def login_post():
    # 요청 바디에서 (id=value&pw=value) 데이터 가져오기
    user_id = request.form.get("id")
    user_pw = request.form.get("pw")

    # 로그인 검증 후 성공 여부와 메시지를 result.html 템플릿에 전달
    if user_id == USER_DB["id"] and user_pw == USER_DB["pw"]:
        return render_template(
            "result.html",
            success = True,
            user_id = user_id,
            method = "POST",
            message = "주소창(URL)에 입력한 id와 pw가 보이지 않고 body에 실려 전송됩니다"
        )
    else:
        return render_template("result.html", success=False)

if __name__ == "__main__":
    app.run()
```


5주차정리) 응용실습1- Flask 템플릿을 이용한 웹 사이트 로그인

■ 응용실습1) flask_methods.py를 템플릿 구조로 변경하기

2) result.html (파이썬 본문과 웹 화면을 참조해서 jinja2 빈칸을 완성하세요)

```
<!DOCTYPE html>
<html lang = "ko">
<head>
  <meta charset = "UTF-8">
  <title>웹사이트 로그인결과</title>
</head>
<body>
```

```
  <h2>Flask 템플릿과 템플릿엔진 jinja2를 사용한 로그인 결과</h2>
  <hr>
  <!-- (빈칸1) id와 pw 검증결과 성공인 경우 ---->
  {% if success %}
    <!-- (빈칸2) method 방식과 user_id 받아서 출력하기 ---->
    <h2>{{ method }} 방식 로그인 성공! 안녕하세요 {{ user_id }}님 </h2>
    <!-- (빈칸3) message 받아서 출력하기 ---->
    <p>{{ message }}</p>
  <!-- (빈칸4) if ~ else, id와 pw 검증결과 실패인 경우 ---->
  {% else %}
    <h2>로그인 실패</h2>
    <p>아이디 또는 비밀번호가 틀렸습니다</p>
  <!-- (빈칸5) if 제어문 종료하기 ---->
  {% endif %}
  <br>
  <a href="/">돌아가기</a>
```

```
</body>
</html>
```

← → ↻ localhost:5000/get/?id=admin&pw=1234	← → ↻ localhost:5000/post/
Flask 템플릿과 템플릿엔진 jinja2를 사용한 로그인 결과	Flask 템플릿과 템플릿엔진 jinja2를 사용한 로그인 결과
GET 방식 로그인 성공! 안녕하세요 admin님	POST 방식 로그인 성공! 안녕하세요 admin님
주소창(URL)에 입력한 id와 pw가 그대로 보입니다	주소창(URL)에 입력한 id와 pw가 보이지 않고 body에 실려 전송됩니다
돌아가기	돌아가기

Jinja2가 사용된 부분

6주차 부터는...

“6주차부터 본격적으로 IoT의 세계로 들어가 봅시다!”

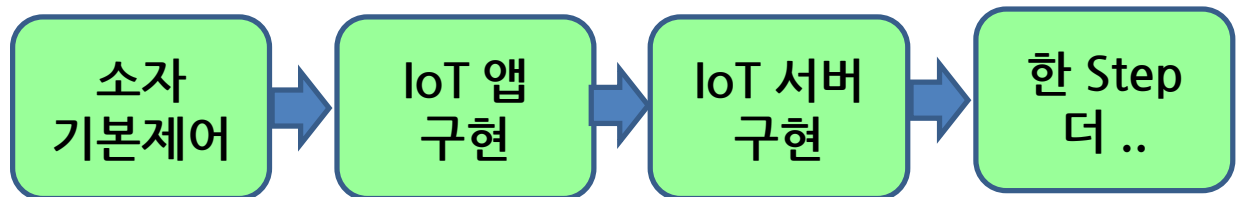
IoT(사물인터넷)의 기본은 네트워크입니다

실시간으로 서버와 클라이언트간 데이터를 주고받으며 상호작용하게 하는 핵심 기반입니다

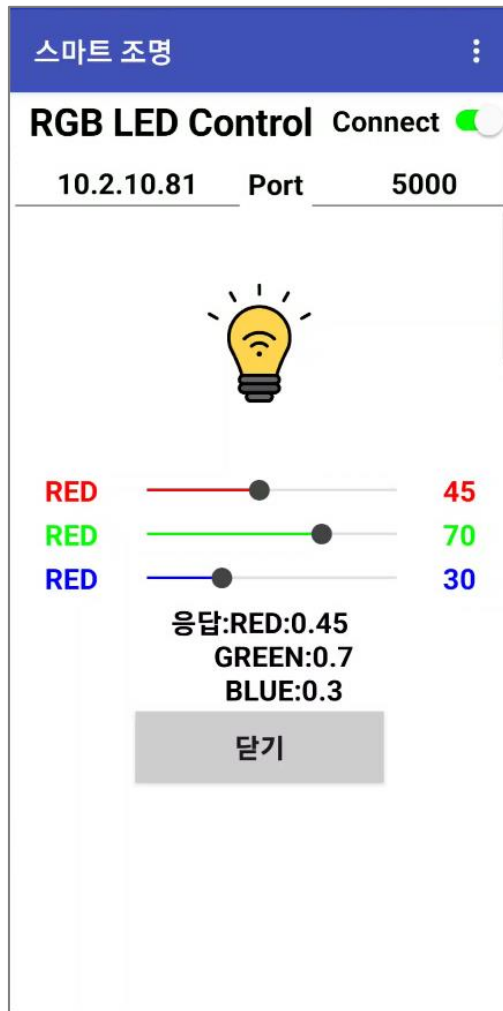
라즈베리파이에 Flask 웹 서버를 구현하고, 소자 제어의 핵심로직을 이용해 웹 서비스를 제공하고자 합니다

그럼, 이제 본격적으로 IoT 의 세계로 들어가 봅시다.

앞으로 이렇게 수업을 합니다.



6주차. 스마트 조명



“6주차에는 “스마트조명” 앱을 제작하여
앱으로 라즈베리파이에 연결된
RGB LED의 밝기를 제어해 보시다”

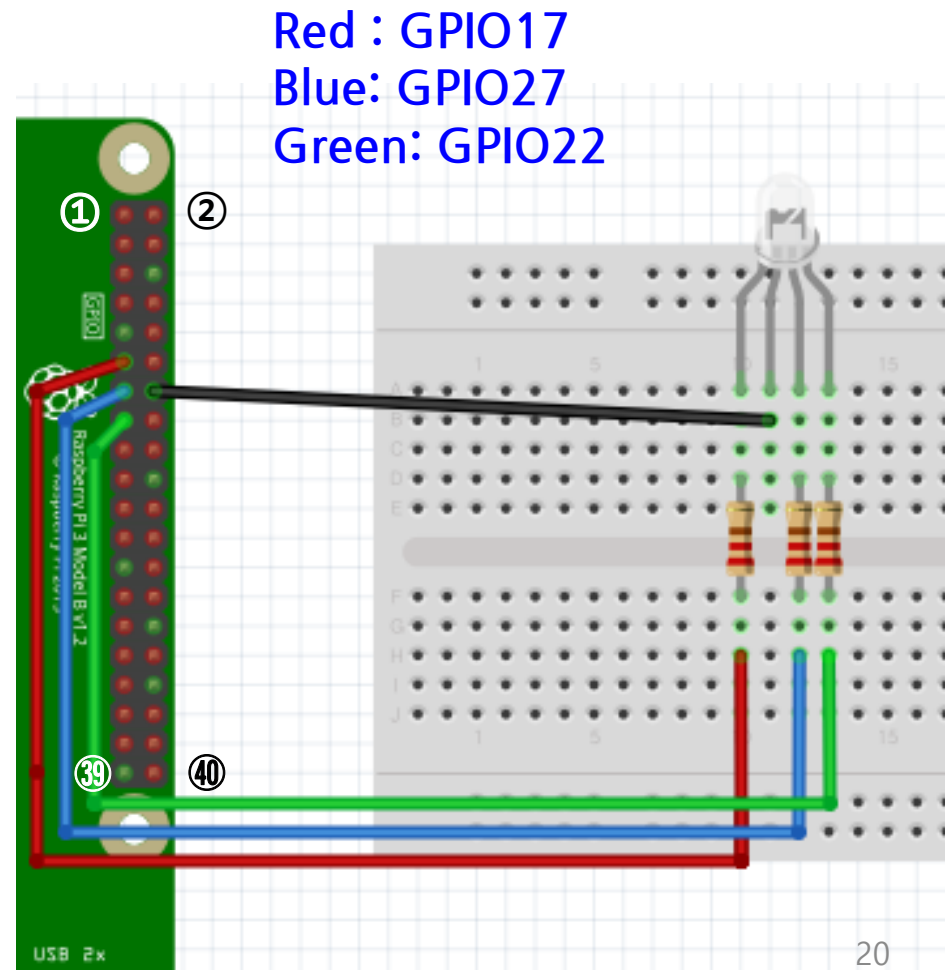
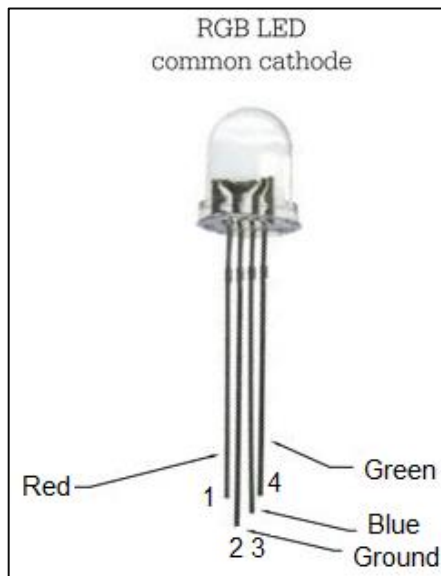


그림참조:<https://www.ledsupply.com/blog/wp-content/uploads/2021/06/Dimmable-LED-Lights.png>

수업준비) RGB LED 회로 구성

□ 수업 시작 전에 rgb.py 회로를 구성하고 실행해서 LED 정상동작 확인해 주세요

라즈베리파이		3색 LED
GPIO#17 (11번)	저항	1 (Red)
GND(14번)		2(GND)
GPIO#27 (13번)	저항	3(Blue)
GPIO#22 (15번)	저항	4(Green)



수업준비) RGB LED 제어로직 코드

□ 실습1: Red → Green → Blue 순서대로 1초간격으로 불이 들어오도록 코딩하기

파일명: rgb.py

```
# RGBLED 기본제어 방법:blink동작
from gpiozero import RGBLED
import time

# RGBLED pin 지정후 객체 생성
rgb = RGBLED(red=17, green=22, blue=27)

print("RGB LED 순서대로 blink하기")
try:
    while True:
        print("RED ON")
        rgb.color = (1, 0, 0) # (r, g, b)
        time.sleep(1)
        print("GREEN ON")
        rgb.color = (0, 1, 0)
        time.sleep(1)
        print("BLUE ON")
        rgb.color = (0, 0, 1)
        time.sleep(1)
except KeyboardInterrupt:
    pass
finally:
    # 예외처리 종료시 모든 리소스 cleanup 하기
    rgb.close()
    print("Program End")
```

1. gpiozero 라이브러리의 RGBLED 클래스이용

- RGBLED 모듈을 쉽게 제어할 수 있도록 만든 전용 도구
- 핵심 제어 수단: color 속성(Property)
- 기본적으로 softPWM으로 동작
전류의 양을 미세 조정하여 밝기조절

2. rgb.color = (값) 형식,

* color는 속성(property)

함수로 호출하는 것이 아니라, 일반 변수처럼 값을 직접 읽거나 바꿀 수 있도록 설계

- 데이터 형태: 3개의 소수점을 가진 튜플 형태 (Red, Green, Blue)
- 값의 범위: 각 색상 채널의 범위는 0.0(완전히 꺼짐)부터 1.0(최대 밝기)까지

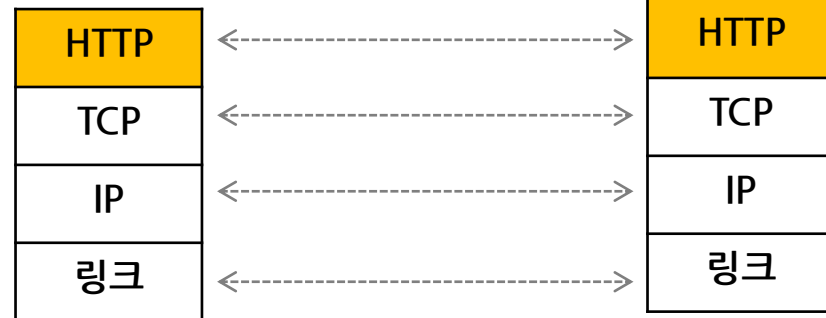
1. 스마트 조명 - 프로토콜 스택

스마트 조명 프로토콜 스택

TCP/IP 프로토콜 스택

클라이언트 앱 / 웹 브라우저

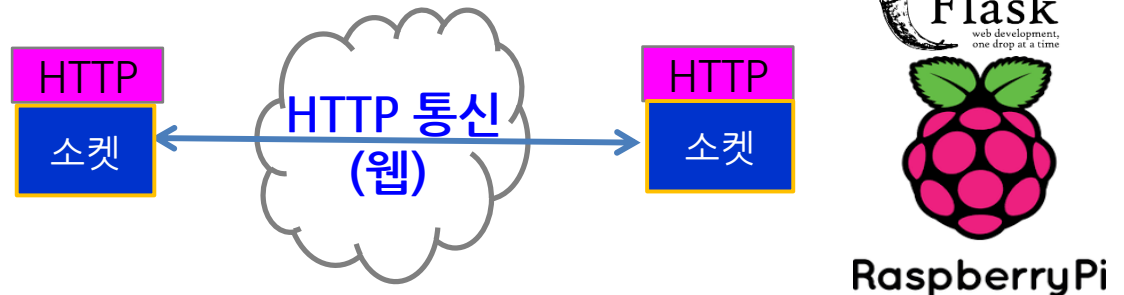
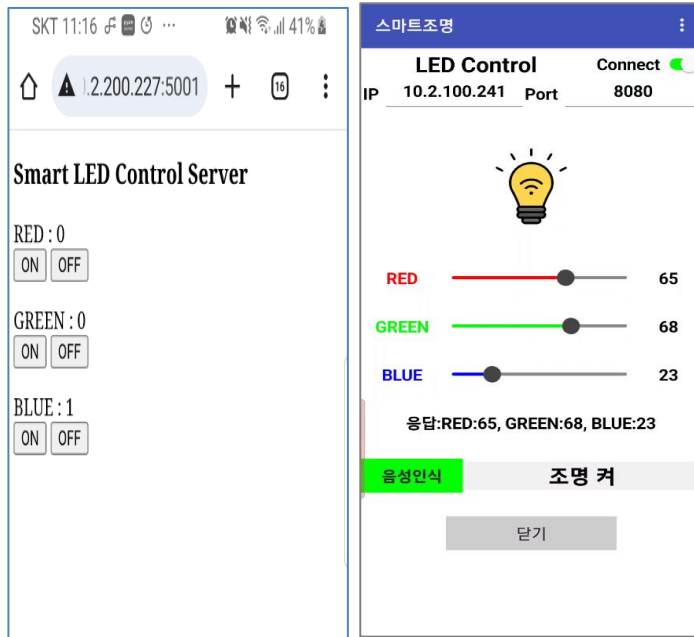
라즈베리파이 웹 서버



“스마트 조명을
2가지 방식으로 구현”

SSR방식

CSR방식



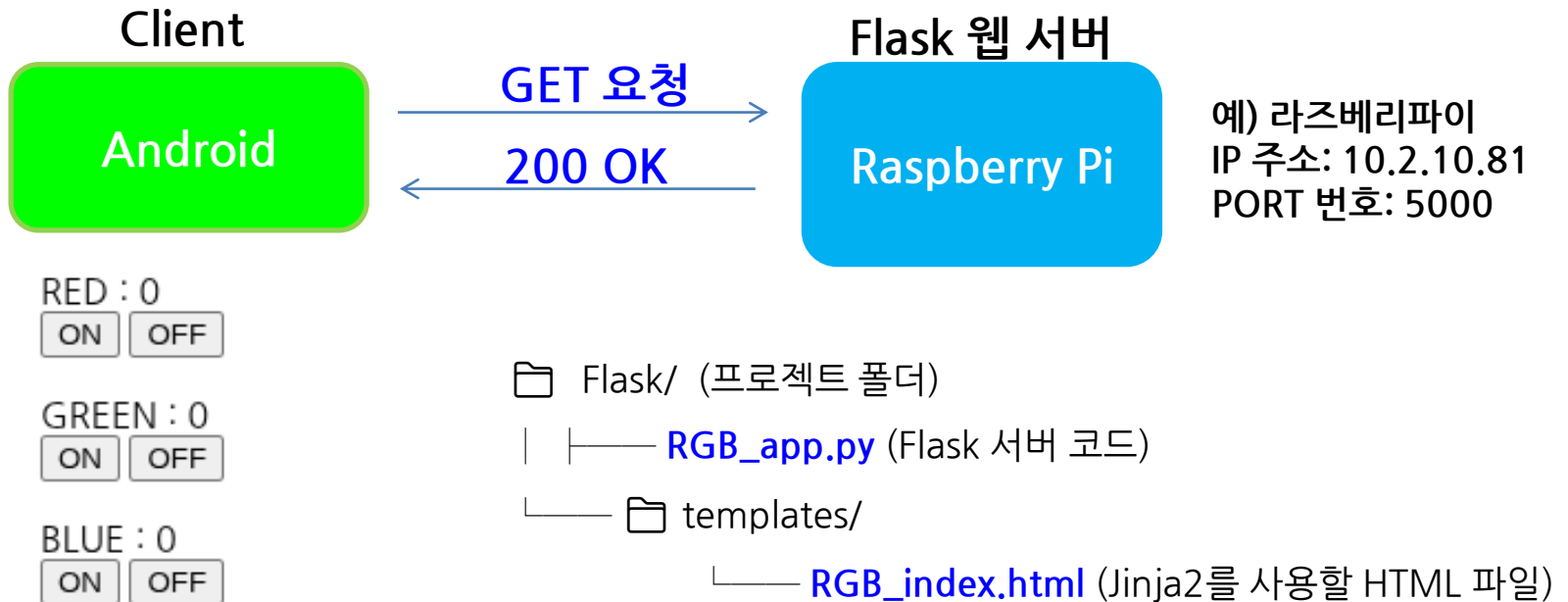
2. 스마트 조명 실습 - 1) SSR 방식

□ 실습1) SSR Server-Side Rendering 방식으로 라즈베리파이 서버 구현

[동작 개요]

- SSR 방식 : 서버가 데이터가 반영된 완성된 HTML 페이지를 만들어서 브라우저에 전달하는 방식
- **GET 방식 사용** : 클라이언트가 파라미터를 URL 쿼리스트링으로 보내 요청하는 방식

`http://라즈베리파이 IP:5000/get?color=RED&state=1`

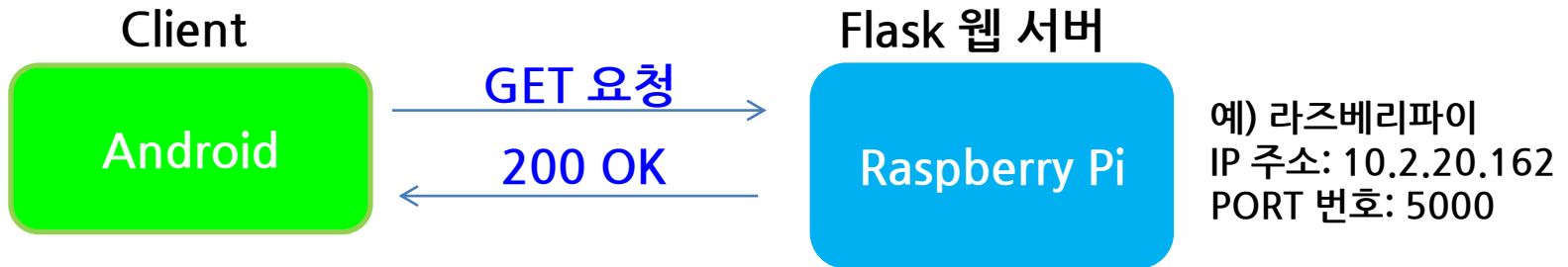


2. 스마트 조명 실습 - 1) SSR 방식

□ 실습1) SSR Server-Side Rendering 방식으로 라즈베리파이 서버 구현

[동작 절차]

② `http://라즈베리파이 IP:5000/get?color=RED&state=1`



```
RGB_state_dict = {'RED': 0, 'GREEN': 0, 'BLUE': 0}
```

- ③ query string parsing
`{'color': 'RED', 'state': '1'}`
- ④ `color = request.args.get('color')`
`state = request.args.get('state')`
- ⑤ # 클라이언트 정보를 받아 RGB 색상과 제어정보 업데이트
`def update_RGB( ):`

2. 스마트 조명 실습 - 1) SSR 방식: 웹 서버 구현

□ 실습1) SSR Server-Side Rendering 방식으로 라즈베리파이 웹 서버 구현 - 서버 코드

◦ 이름_6주차_RGB_app.py

```
# 스마트조명 웹 서버 프로그램
from flask import Flask, request, render_template
from gpiozero import RGBLED

# 웹 서버 객체 생성
app = Flask(__name__)

# RGBLED pin설정 및 객체 생성
rgb = RGBLED(red=17, green=22, blue=27)

# RGB 상태 DB (딕셔너리)
RGB_state_dict = {'RED': 0, 'GREEN': 0, 'BLUE': 0}

# RGBLED 제어 핵심로직
def update_RGB():
    r_val = float(RGB_state_dict['RED'])
    g_val = float(RGB_state_dict['GREEN'])
    b_val = float(RGB_state_dict['BLUE'])
    rgb.color = (r_val, g_val, b_val)  # rgb.color 사용해서 R,G,B PWM값 지정
    update_RGB()  # RGBLED 초기화
```

RGB 핵심로직(PWM)

* `strict_slashes=False`는 URL 끝에 붙는 슬래시(/)를 엄격하게 구분하지 않겠다는 설정
(URL 끝에 슬래시가 있든 없든 두 경우 모두 정상적으로 같은 페이지(라우트)로 연결)

```
# 루트로 접속 시 처리
@app.route('/', strict_slashes=False)
def index():
    return render_template('RGB_index.html', RGB_states=RGB_state_dict)

# SSR방식. GET 요청에 대한 처리 (웹 브라우저용)
@app.route('/get', methods=['GET'], strict_slashes=False)
def get_RGB():
    # URL 쿼리에서 key로 value 저장하기
    color = request.args.get('color')
    state = request.args.get('state')
    색상(키)에 PWM(값) 저장 = int(state)
    update_RGB()  # RGBLED 상태 업데이트
    return render_template('RGB_index.html', RGB_states=RGB_state_dict)

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

템플릿엔진으로 넘겨주는 값

'0.0.0.0'은 "모든 IP 주소"를 나타내는 특별한 예약 주소

2. 스마트 조명 실습 - 1) SSR 방식: HTML 구현

□ 실습1) SSR Server-Side Rendering 방식으로 라즈베리파이 웹 서버 구현 - HTML 코드

- templates/RGB_index.html

- 메인 코드에서 RGB_states_dict를 RGB_states로 전달 받음

```
return render_template('RGB_index.html', RGB_states=RGB_state_dict) # 상태 저장 딕셔너리
RGB_state_dict = {'RED': 0, 'GREEN': 0, 'BLUE': 0}
```

```
<!DOCTYPE html>
<html>
<head>
  <!-- 스마트폰 화면 크기에 맞춰 웹페이지를 맞춰주는 필수 태그 -->
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>LED</title>
</head>
<body>
  <h3>Smart LED Control Server</h3>

  <!-- Jinja2 반복문 시작 -->
  {% for color, state in RGB_states.items() %}
  <p>
    <!-- 딕셔너리의 key(색상)과 value(상태)를 받아 화면에 출력 -->
    {{ color }} : {{ state }}
    <br>
    <!-- 버튼 클릭시 주소창에 파라미터를 담아 서버로 전송 -->
    <a href="/get?color={{ color }}&state=1"><button>ON</button></a>
    <a href="/get?color={{ color }}&state=0"><button>OFF</button></a>
  </p>
  {% endfor %}
  <!-- jinja2 반복문 끝 -->

</body>
</html>
```

파이썬 딕셔너리(RGB_states)에 저장된 모든 key(키)와 value(값)의 쌍을 하나씩 꺼내어 반복문(for문)을 돌리겠다는 의미

Smart LED Control Server

RED : 1

ON OFF

GREEN : 0

ON OFF

BLUE : 0

ON OFF

Jinja2가 사용된 부분

2. 스마트 조명 실습 - 1) SSR 방식

□ 실습1) SSR Server-Side Rendering 방식으로 라즈베리파이 웹 서버 구현

◦ 실행하기

http://라즈베리파이 IP:5000/get?color=RED&state=1

Smart LED Control Server

RED : 1

GREEN : 0

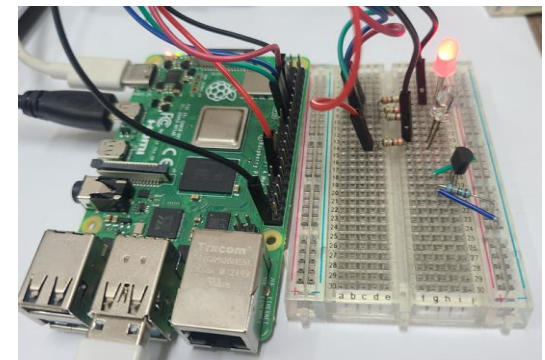
BLUE : 0

GET 요청

200 OK

Flask 웹 서버
(app.py + HTML)

Raspberry Pi



3. 스마트 조명 - 2) CSR 방식

□ 실습2) CSR Client-Side Rendering 방식으로 라즈베리파이 서버 구현

CSR방식은 클라이언트가 웹 화면을 직접 만드는 방식으로 클라이언트 앱은 앱 인벤터를 이용해 제작합니다.

Flask 웹 서버는 SSR 방식의 GET 방식으로 웹 브라우저가 전송한 파라미터 데이터를 받아 제어해 보았습니다.

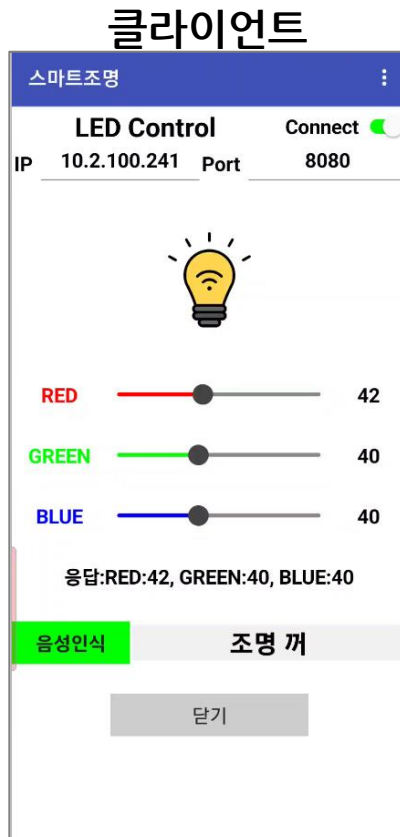
이젠 CSR 방식으로 앱에서 POST 방식으로 색상과 PWM 데이터를 받아서 RGB를 제어해 봅시다.

- 기존 RGB_app.py에 POST 방식 처리를 위해 라우팅을 추가해야 합니다.

3. 스마트 조명 - 2) CSR 방식: 웹 서버 구현

□ 실습2) CSR Client-Side Rendering 방식으로 라즈베리파이 웹 서버 구현

- CSR 방식 : 브라우저(클라이언트)가 웹 화면을 직접 만드는 방식으로, 서버는 순수한 데이터(JSON 혹은 텍스트)만 전송하는 방식
- POST 요청 : 모든 파라미터를 요청메시지 바디에 실어 요청하는 방식



http://라즈베리파이 IP주소:5000

POST 요청

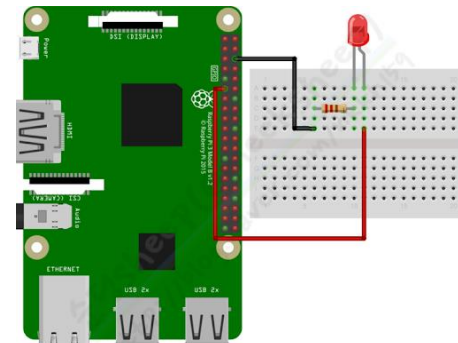
요청메시지 body
"color=RED&pwm_value=66"

200 OK

응답메시지
"RED:66, GREEN:0, BLUE:0"

Flask 웹 서버
(app.py)

Raspberry Pi



3. 스마트 조명 - 2) CSR 방식: 웹 서버 구현

□ 실습2) CSR Client-Side Rendering 방식으로 라즈베리파이 웹 서버 구현

- 이름_6주차_RGB_app.py 에 POST 방식 처리를 위한 라우팅 추가 구현 (완성하기)

```
# CSR방식. POST 요청에 대한 처리(앱 인벤터 제어용)
@app.route('/post', POST 방식 추가, strict_slashes=False)
def post_RGB():
    # 요청메시지 바디에서 key로 value 저장
    color = request.form.get('color')
    pwm_v = request.form.get('pwm_value')

    해당 색상(키)에 PWM(값) 저장 = int(pwm_v)/100
    update_RGB() # RGBLED pwm 값 업데이트

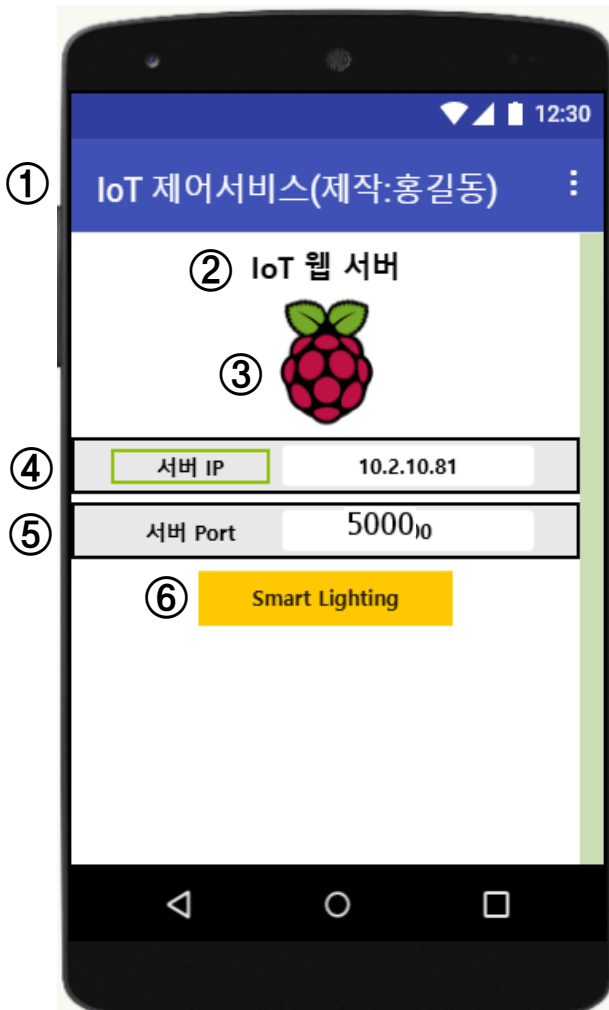
    # 앱 인벤터가 성공여부를 확인할 수 있도록 결과 문자열 반환
    return f"""RED:{RGB_state_dict['RED']},
    GREEN:{RGB_state_dict['GREEN']}
    BLUE:{RGB_state_dict['BLUE']}"""
```

3. 스마트 조명 - 2) CSR 방식: 클라이언트 앱 제작

□ Screen1 디자인하기

프로젝트이름 : IoT_System_학번

Screen1

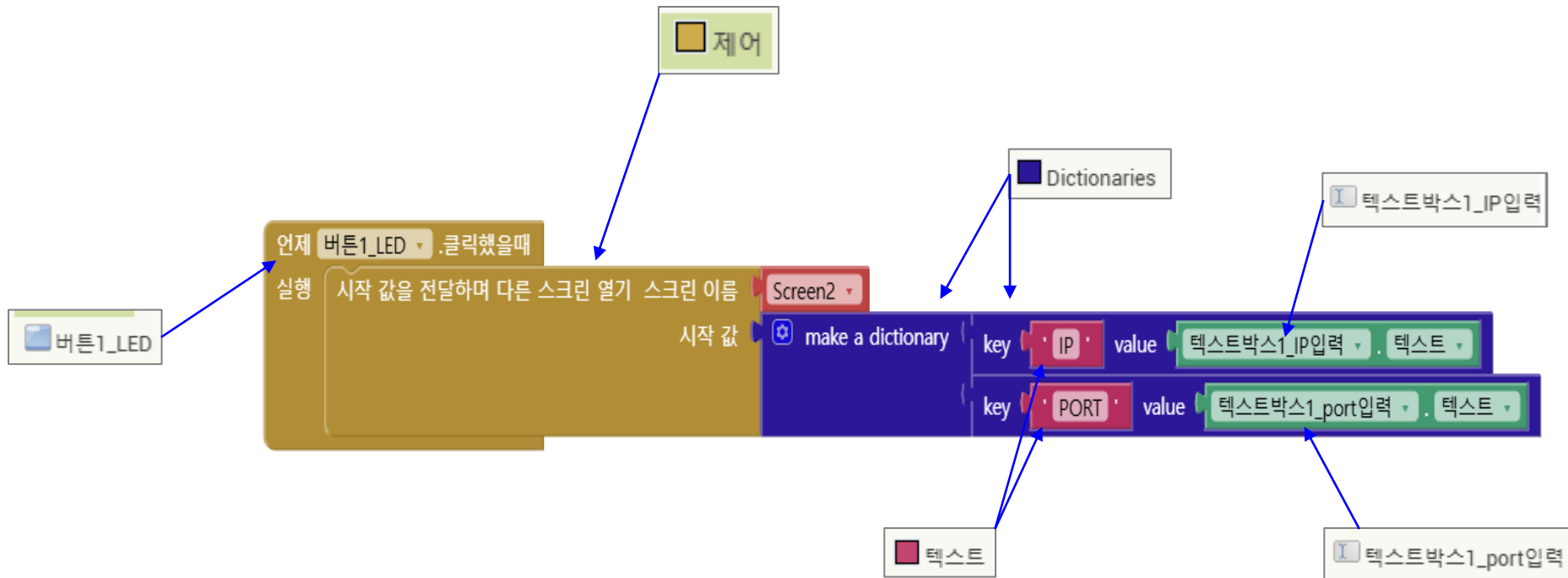


	컴포넌트	이름	속성
①	Screen	Screen1	수평정렬-가운데, 제목 -IoT 제어서비스(제작:이름) 스크롤가능여부 -체크
②	레이블	레이블_title	글꼴굵게, 글꼴크기-20, 너비 - 부모요소 텍스트 - IoT 웹 서버, 텍스트정렬- 가운데
③	이미지	이미지1	높이-20%, 너비-20% 사진 - images.png (RPI 이미지 미리 준비하기)
④	수평배치	수평배치1	수평, 수직정렬-가운데, 너비-부모요소
	레이블	레이블_서버IP	글꼴굵게, 글꼴크기-15, 너비 -30% 텍스트 - 서버 IP, 텍스트정렬- 가운데
	텍스트박스	텍스트박스_IP입력	글꼴굵게, 글꼴크기-15, 너비 - 50% 텍스트 - 라즈베리파이IP주소, 텍스트정렬- 가운데
⑤	수평배치	수평배치1	수평, 수직정렬-가운데, 너비-부모요소
	레이블	레이블_서버Port	글꼴굵게, 글꼴크기-15, 너비 -30% 텍스트 - 서버 Port, 텍스트정렬- 가운데
	텍스트박스	텍스트박스_Port입력	글꼴굵게, 글꼴크기-15, 너비 - 50%, 숫자만, 텍스트 - 5000, 텍스트정렬- 가운데
⑥	버튼	버튼1_LED	배경색-주황, 글꼴굵게, 글꼴크기-15, 너비 -50% 모양-직사각형, 텍스트 - Smart Lighting, 텍스트정렬- 가운데

3. 스마트 조명 - 2) CSR 방식: 클라이언트 앱 제작

□ Screen1 코딩하기

- 앱 디자인에서 Screen2 미리 추가해 놓기 (스크린 추가 >> Screen2)
- Screen1 블록 화면에서 앱 코딩



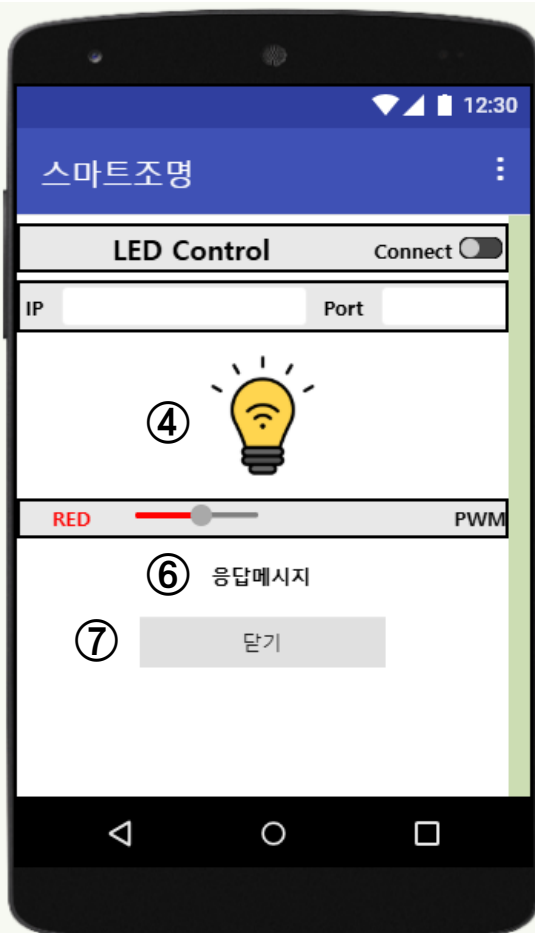
코드동작: LED 버튼을 클릭 했을 때 Screen2로 앱 화면이 전환됨
이때, 사용자가 텍스트박스에 입력한 IP주소와 Port번호를 시작 값으로 전달함

3. 스마트 조명 - 2) CSR 방식: 클라이언트 앱 제작

❑ Screen2 디자인하기 - RED part 우선 구현하기

Screen2

	컴포넌트	이름	속성
①	Screen	Screen2	수평정렬-가운데, 스크롤가능여부- 체크 제목 - 스마트조명
②	수평배치	수평배치1	수평, 수직정렬-가운데, 너비-부모요소
	레이블	레이블1_title	글꼴굵게, 글꼴크기-20, 너비 - 부모요소 텍스트- LED Control, 텍스트정렬- 가운데
	스위치	스위치1	글꼴굵게, 글꼴크기-15, 텍스트 - Connect
③	수평배치	수평배치1	수평, 수직정렬-가운데, 너비-부모요소
	레이블	레이블1_IP	글꼴굵게, 글꼴크기-15, 텍스트- IP, 정렬-가운데
	텍스트박스	텍스트박스1_IP	글꼴굵게, 글꼴크기-15, 정렬- 가운데
	레이블	레이블1_Port	글꼴굵게, 글꼴크기-15, 텍스트- Port, 정렬-가운데
	텍스트박스	텍스트박스1_Port	글꼴굵게, 글꼴크기-15, 정렬- 가운데, 숫자만
④	이미지	이미지1	높이-25%, 너비-25% 사진 - 조명 이미지 (이미지는 자유로)



보이지 않는 컴포넌트

⑧ 웹1

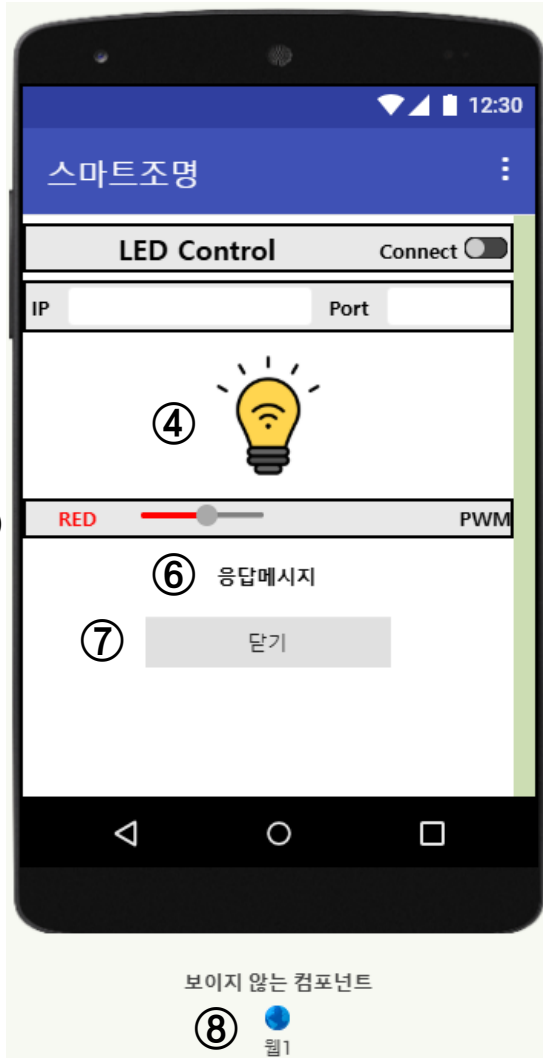
3. 스마트 조명 - 2) CSR 방식: 클라이언트 앱 제작

□ Screen2 디자인하기(계속) - RED part 우선 구현하기

Screen2

	컴포넌트	이름	속성
⑤	수평배치	수평배치1	수평, 수직정렬-가운데, 너비-부모요소
	레이블	레이블1_RED	글꼴굵게, 글꼴크기-15, 너비 - 20% 텍스트- RED, 텍스트색상-빨강, 정렬-가운데
	슬라이더	슬라이더1_RED	왼쪽색상-빨강, 너비-60%, 최댓값-100, 최솟값-0, 섬네일위치-0
	레이블	레이블1_R섬네일	글꼴굵게, 글꼴크기-15, 너비-15%, 텍스트- PWM, 텍스트색상-빨강, 정렬-가운데
⑥	레이블	레이블1_응답메시지	글꼴굵게, 글꼴크기-15 텍스트 - 응답메시지, 텍스트정렬- 가운데
⑦	버튼	버튼1_닫기	글꼴굵게, 글꼴크기-15, 너비 -50% 모양-직사각형, 텍스트 -닫기, 텍스트정렬- 가운데
⑧	웹	웹1	

* 연결 >> 웹



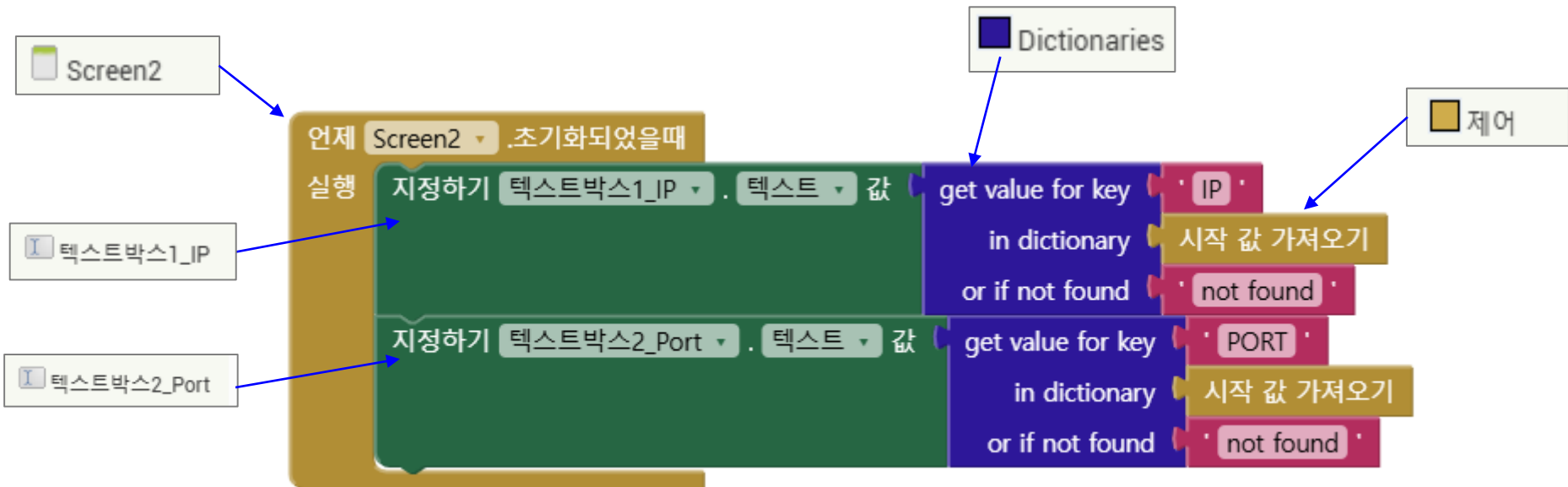
보이지 않는 컴포넌트

⑧ 웹1

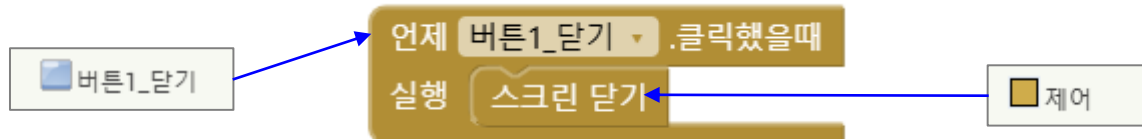
3. 스마트 조명 - 2) CSR 방식: 클라이언트 앱 제작

□ Screen2 코딩하기 - RED part 우선 구현하기

코드동작: 스크린이 초기화 되었을 때(시작했을 때)
Screen1으로 부터 시작값(IP와 Port정보)를 가져와서 출력하기



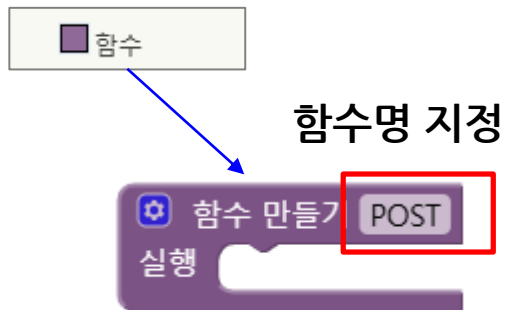
코드동작: 닫기 버튼을 클릭 했을 때, Screen2 화면을 닫고 Screen1(홈화면)으로 돌아가기



3. 스마트 조명 - 2) CSR 방식: 클라이언트 앱 제작

□ 앱 인벤터에서 함수를 사용하는 방법

“블록코딩에서 RED의 동작코드를 그대로 복사해서 GREEN과 BLUE에 사용해도 되지만 이런 경우 코드 중복이 됩니다. 이 경우 함수를 사용해 봅시다 ”



함수에 매개변수 추가하기 (color, pwm_value)



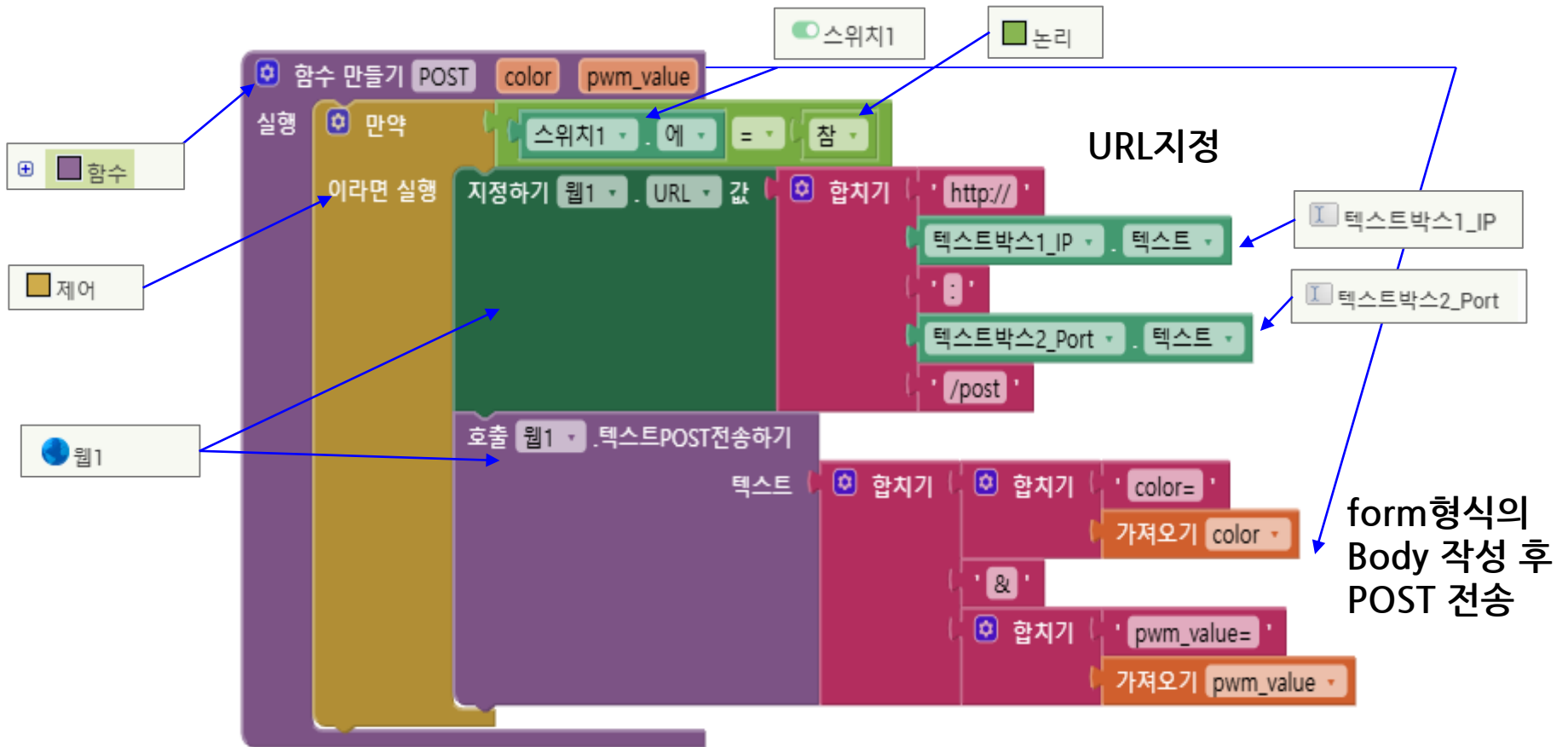
함수 생성



3. 스마트 조명 - 2) CSR 방식: 클라이언트 앱 제작

□ Screen2 코딩하기(계속) - RED part 우선 구현하기

코드동작: 섬네일의 위치가 변경되었을 때마다 [웹] POST 방식으로 파라미터 전송하기



바디의 form형식: "color=RED&pwm_value=90"

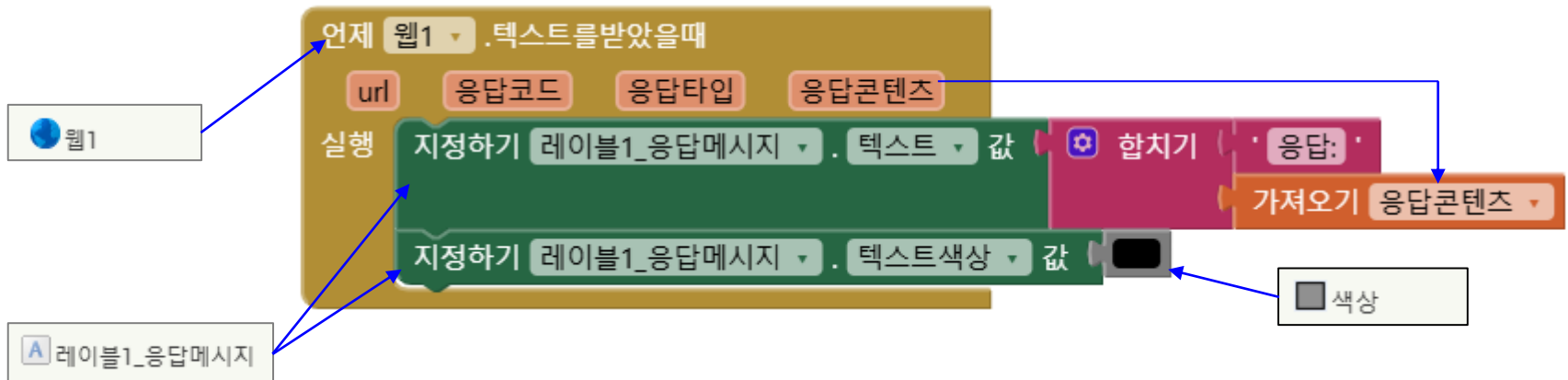
3. 스마트 조명 - 2) CSR 방식: 클라이언트 앱 제작

□ Screen2 코딩하기(계속) - RED part 우선 구현하기

코드동작: 슬라이더_RED 섬네일 위치 변경 시 POST 함수 호출



코드동작: 서버의 응답메시지 수신 후 레이블에 출력



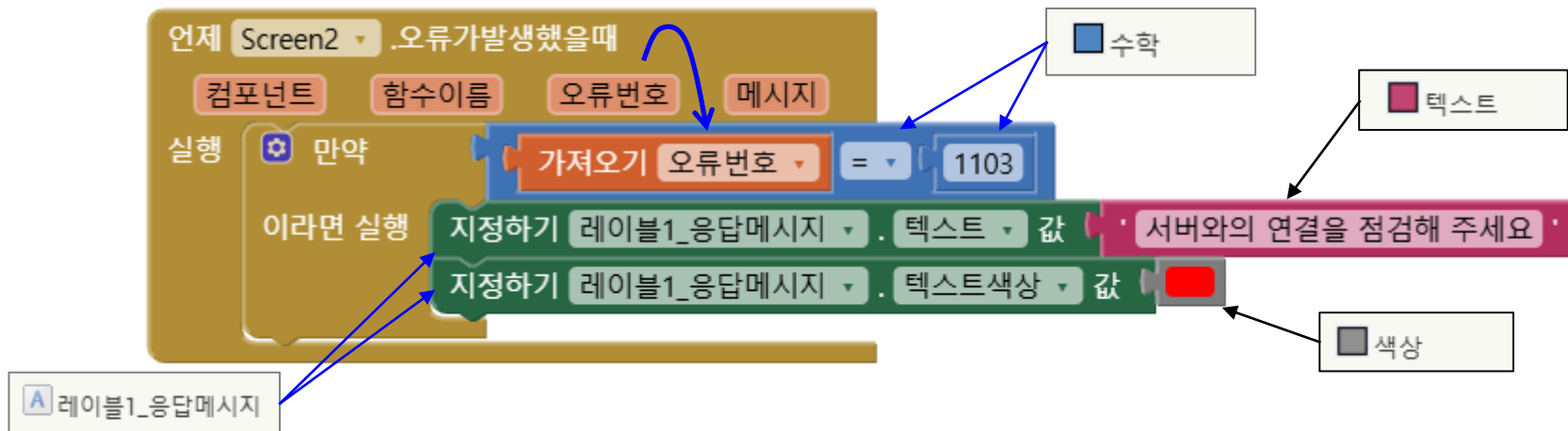
3. 스마트 조명 - 2) CSR 방식: 클라이언트 앱 제작

❑ Screen2 코딩하기(계속) - RED part 우선 구현하기

코드동작: 서버와의 연결에 실패하는 경우 오류 메시지 출력

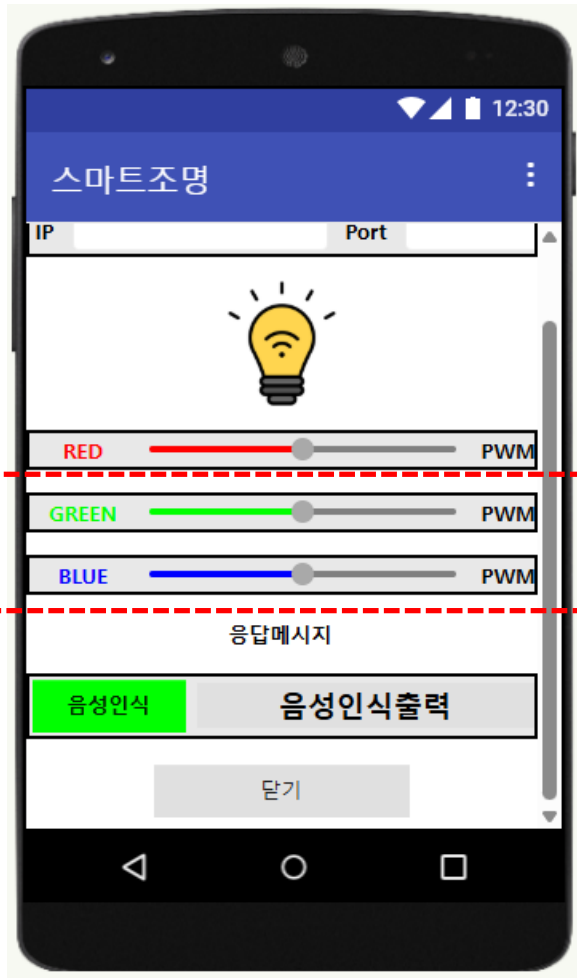
※ Error 1103:
네트워크 연결 문제로 인해 클라이언트에서
서버로 연결할 수 없음을 의미

Error 1103: Unable to complete the given request with the text "key=f7ba16234fc95192ddbfaad8e9a97a2c37735aed&ipaddr=192.168.1.42&port=9987&webrtc=true&version=2.62&api=30&installer=com.android.vending&aid=a9b770da-062c-4017-bff8-cc35e7221d40&r2=true" with the specified URL: http://rendezvous.appinventor.mit.edu/rendezvous/



3. 스마트 조명 - 2) CSR 방식: 클라이언트 앱 제작

❑ Screen2 디자인 - GREEN, BLUE part 추가 디자인



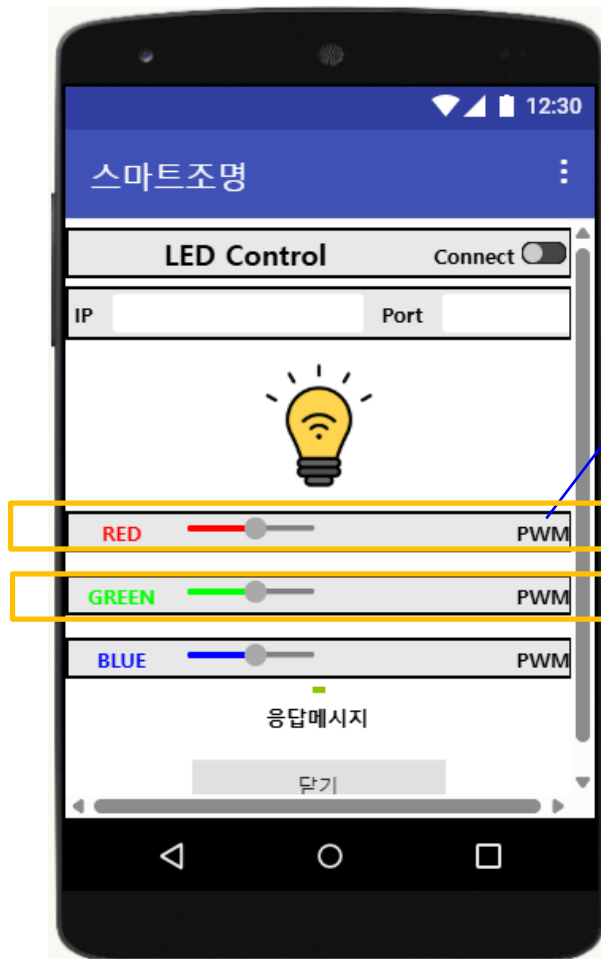
RED가 정상동작 한다면

스마트조명 앱에서 GREEN과 BLUE를 추가하여
RGB 모두 제어해 봅시다

방식은 RED 제어와 동일합니다
(이미 제작된 RED 파트를 이용해 쉽게 디자인과
코딩을 할 수 있습니다)

33. 스마트 조명 - 2) CSR 방식: 클라이언트 앱 제작

□ Screen2 디자인 - GREEN, BLUE 제어 추가 디자인



수평배치	수평배치1	수평, 수직정렬-가운데, 너비-부모요소
레이블	레이블1_RED	글꼴굵게, 글꼴크기-15, 너비 - 20% <u>텍스트- RED, 텍스트색상-빨강</u> , 정렬-가운데
슬라이더	슬라이더1_RED	<u>왼쪽색상-빨강</u> , 너비-60%, 최댓값-100, 최솟값-0, <u>섬네일위치-0</u>
레이블	레이블1_R <u>섬네일</u>	글꼴굵게, 글꼴크기-15,너비-15%, <u>텍스트- PWM, 텍스트색상-빨강</u> , 정렬-가운데

동일한 방법으로 GREEN, BLUE 추가하기
컴포넌트 이름과, 속성의 붉은색 부분만 변경

Ctrl-C/Ctrl-V 후에 필요한 부분만 변경

Tip) - RED part 전체를 선택 한 후 Ctrl-C/Ctrl-V
- 레이아웃 조정하기
- 컴포넌트의 이름과 색상 변경
* 앱 코드 부분도 같이 생성됨

보이지 않는 컴포넌트



월1

3. 스마트 조명 - 2) CSR 방식: 클라이언트 앱 제작

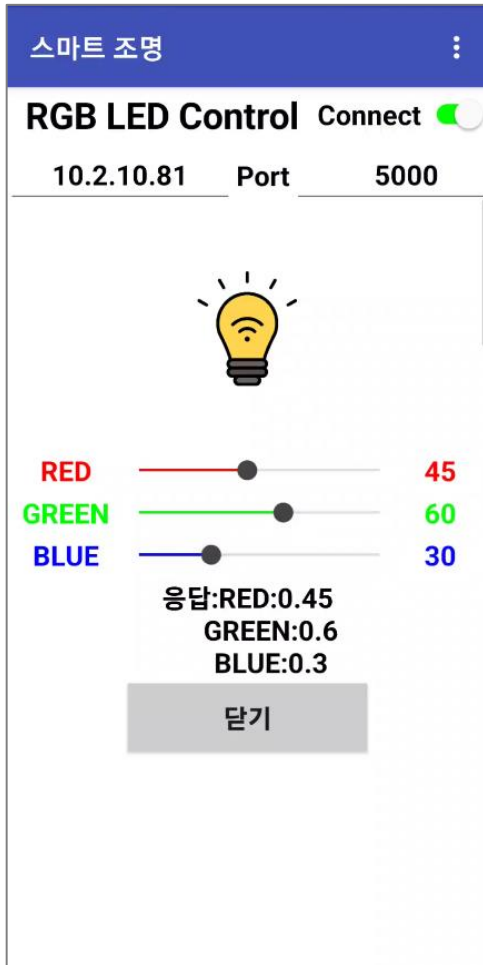
□ Screen2 앱 디자인 - GREEN, BLUE part 추가 코딩

GREEN과 BLUE
동작코드 추가



3. 스마트 조명 - 2) CSR 방식

□ 스마트조명 실행하기



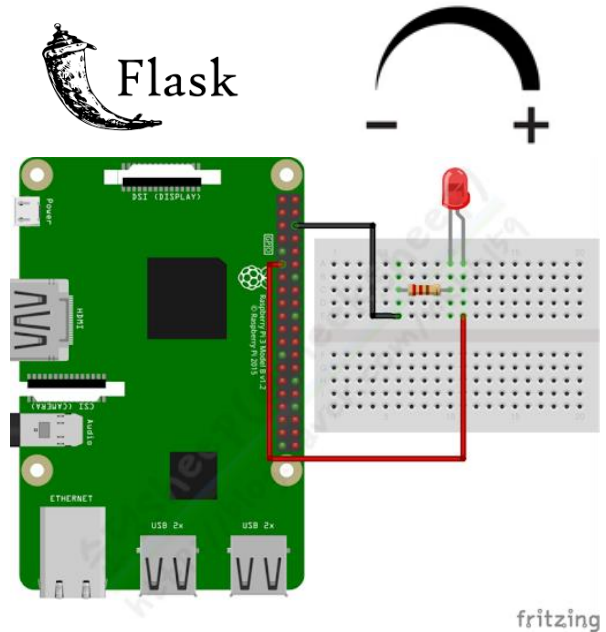
“RGB LED PWM 동작 확인하기”

POST 방식

“color=GREEN&pwm_value=60”

200 OK

“RED:0.45, GREEN:0.6, BLUE:0.3”



6주차 과제 제출 안내

□ 6주차 실습과제

- 과제: 3교시 실습한 스마트 조명 실습 파일을 제출해 주세요
- 제출파일: '이름_6주차_RGB_app.py', 'IoT_System_학번.aia'

** 실습 시 잘 안된 부분이 있다면 comment를 남겨 주세요*



수고 많았습니다!